

From Narrative Documents to Compiled Knowledge: A Case Study of Knowledge Preservation Across PDF, JSON, Markdown and YAML CKF Representations

Paulo Tomazinho, PhD
CKF Research

Manuscript type: empirical case study and software artifact analysis
Date: May 21, 2026

Abstract

This paper investigates how much useful knowledge is preserved when a short narrative PDF manifesto is converted into structured Compiled Knowledge Format (CKF) representations in JSON, Markdown and YAML. The study compares four artifacts derived from the same conceptual source: a Portuguese narrative PDF, a Portuguese CKF JSON package, an English CKF Markdown package and an English CKF YAML package. It also audits the submitted TypeScript source code of the current CKF compiler, including its semantic chunker, per-chunk structured extraction schema, map-reduce reducer, provider adapters and quality gates. The empirical artifact comparison shows a strong format/function trade-off: JSON, Markdown and YAML improve machine readability, schema orientation and explicit sectioning, but the examined CKF outputs preserve only about 17.5% of a 20-unit substantive knowledge rubric derived from the PDF. Most losses concern the very elements that motivate CKF: composition failures, implicit procedural dependencies, the relationship to RAG/GraphRAG/MCP, provenance as audit infrastructure, confirmatory benchmarking and falsifiability. The code audit indicates that the current compiler design (CKF v1.0) contains mechanisms that could mitigate these losses - `source_refs`, `source_excerpts`, confidence scoring, stable span IDs and quality metrics - but the observed output artifacts appear to follow an older or shallower CKF pattern. The central conclusion is that structured format alone is insufficient: useful knowledge preservation depends on extraction fidelity, source-grounded traceability and semantic completeness criteria. The paper proposes an evaluation framework for CKF compilation quality and recommends preservation-aware quality gates before structured knowledge packages are used in retrieval-augmented generation or agentic systems.

Keywords: Compiled Knowledge Format; RAG; GraphRAG; knowledge representation; provenance; structured extraction; hallucination; source attribution; JSON; YAML; Markdown; PDF

1. Introduction

Large language model (LLM) agents increasingly read, retrieve, summarize and act on documents originally designed for human readers. This creates a representational mismatch: PDF, DOCX and prose-heavy Markdown documents encode meaning through narrative flow, visual layout, implicit scope and shared human conventions, whereas LLM-based systems consume text spans, retrieved chunks, tool schemas and context windows. The question motivating this study is therefore practical and falsifiable: when a narrative PDF is converted into structured CKF files, how much useful knowledge is actually preserved?

The study is grounded in a single, intentionally small corpus: the CKF manifesto PDF and three structured CKF derivatives in JSON, Markdown and YAML. The narrow corpus makes the study a case study, not a general benchmark. Its value is diagnostic: because all artifacts are short and conceptually aligned, losses are

visible at the level of individual claims, relations and design commitments. The study also uses the submitted compiler source code to distinguish two questions that are often conflated: whether CKF as a target schema is useful, and whether a particular compilation run produced a faithful knowledge package.

The main finding is a negative but useful result. The structured artifacts are substantially more machine-readable than the PDF, but they are semantically shallow relative to the manifesto. They preserve the existence of a CKF package and a generic “intellectual thesis,” while omitting many of the source’s core propositions. This finding should not be read as evidence against CKF in general. Rather, it supports a stronger claim: a CKF compiler must be evaluated not only by schema validity or structural density, but by semantic preservation against the source.

2. Background and Related Work

2.1 Retrieval-augmented generation and structured retrieval

Retrieval-augmented generation (RAG) connects parametric language models with external non-parametric memory, allowing generated answers to depend on retrieved evidence rather than only model weights (Lewis et al., 2020). Later work has expanded this direction through adaptive retrieval and critique, such as Self-RAG, which trains models to retrieve and reflect on evidence when needed (Asai et al., 2024). GraphRAG extends retrieval beyond local passage lookup by building graph-based representations and community summaries for global sensemaking over corpora (Edge et al., 2024). CKF is aligned with this trajectory but addresses a different layer: the structure of the knowledge units that retrieval systems carry.

Long-context models do not remove the need for structured representations. Liu et al. (2024) show that models can fail to use information reliably when relevant evidence is located in the middle of long contexts. This is relevant to CKF because a knowledge package can make relations, exceptions and procedural dependencies explicit before the retrieval or generation step.

2.2 Hallucination, attribution and provenance

Hallucination research increasingly distinguishes unsupported generation, factual inconsistency and failures of grounding (Huang et al., 2025). The CKF manifesto introduces a narrower operational concept: composition failure, where the answer contradicts available source information because relations among fragments were not reconstructed correctly. This framing is compatible with attribution research. Rashkin et al. (2023) define Attributable to Identified Sources (AIS) as a framework for verifying whether generated claims are supported by provided sources. The W3C PROV family similarly treats provenance as representable, exchangeable information about entities, activities and agents (W3C Provenance Working Group, 2013; Moreau et al., 2015).

For CKF, provenance is not merely a citation layer. The submitted compiler schema requires many extracted items to carry `source_basis`, `confidence`, `source_refs` and `verbatim source_excerpts`. This makes provenance part of the data structure itself, not a post-hoc annotation.

2.3 Document and data formats

PDF is a standardized document format optimized for reliable presentation and exchange (ISO, 2020). JSON is a lightweight, language-independent data interchange format with a small set of syntax rules for structured data (Bray, 2017), and JSON Schema provides a validation vocabulary for structured JSON instances (JSON Schema, 2022). Markdown, as formalized by CommonMark, is a plain text format for structured documents (CommonMark, 2024). YAML is a human-friendly data serialization language widely used for configuration

and hierarchical data (YAML Language Development Team, 2021). These formats are not interchangeable in purpose: PDF favors presentation fidelity, JSON favors programmatic interchange, Markdown favors mixed human-machine editing, and YAML favors readable hierarchical configuration.

3. Research Questions

RQ1. How do the four artifacts differ in physical size, character count, line count and structural density?

RQ2. Which substantive knowledge units from the narrative PDF are preserved in JSON, Markdown and YAML CKF outputs?

RQ3. What information is lost and what structural information is gained in the conversion?

RQ4. Does the submitted CKF compiler architecture contain mechanisms capable of addressing observed losses?

4. Materials and Methods

4.1 Corpus

The corpus contains four files: (1) a Portuguese narrative PDF titled “Why CKF exists - Compiled Knowledge Format”; (2) a Portuguese JSON CKF file; (3) an English Markdown CKF file; and (4) an English YAML CKF file. The analysis also includes eight TypeScript files representing the current compiler implementation: `quality.ts`, `providers-manifest.ts`, `compiler.functions.ts`, `clientHelpers.ts`, `chunker.ts`, `schema.ts`, `reduce.ts` and `providers.ts`.

4.2 Measurements

For each artifact, the study measured extension, byte size, kilobytes, pages when applicable, line count, character count and word count. Structural density was approximated as counted CKF elements per kilobyte, including entities, concepts, contextual triggers, mental models, playbooks, question-answer pairs, retrieval chunks, atomic units and source traceability items. Human and computational legibility were scored on a 1-5 ordinal rubric. These scores are interpretive and intended for comparative diagnosis, not psychometric measurement.

4.3 Knowledge preservation rubric

The PDF was manually decomposed into 20 substantive knowledge units. Each CKF artifact was scored 1.0 when a unit was clearly preserved, 0.5 when partially preserved or generalized, and 0.0 when absent. The resulting preservation score is the mean of the 20 unit-level scores. This method deliberately separates structural validity from semantic fidelity: a CKF package can be well-formed yet fail to preserve the claims that make the source useful.

4.4 Compiler code audit

The code audit examined five compiler subsystems: semantic chunking, provider normalization, structured extraction schema, map-reduce merging and quality scoring. The audit focused on whether the implementation design supports faithful extraction, traceability, deduplication and operational quality control.

5. Results

Table 1. Physical and descriptive artifact metrics.

Format	Ext.	Bytes	KB	Pages	Lines	Chars	Words	CKF elems.	Interpretation
PDF narrative	.pdf	137,685	134.46	3	84	5,862	902	0	Source-original narrative; best human audit trail
JSON CKF	.json	9,474	9.25	-	337	9,472	720	15	Highest API/schema interoperability
Markdown CKF	.md	8,430	8.23	-	352	8,423	831	15	Best mixed human/technical review format
YAML CKF	.yaml	7,460	7.29	-	243	7,455	718	15	Compact hierarchical representation; indentation-sensitive

Table 1 shows that the PDF is by far the largest file in bytes because PDF stores presentation, layout and document overhead. The structured files are much smaller and text-dense. Markdown has the highest line count, reflecting sectioned human-readable presentation. YAML is the most compact of the structured files in this corpus.

Table 2. CKF structural population by format.

Section family	PDF	JSON	Markdown	YAML
entities	0	7	7	7
concepts	0	1	1	1
principles / heuristics / decision_rules / procedures	0	0	0	0
causal_chains / if_then_rules / exceptions	0	0	0	0
contextual_triggers	0	1	1	1
mental_models / playbooks / qa_pairs / retrieval_chunks / atomic_units	0	5	5	5
source_traceability	0	1	1	1
total counted elements	0	15	15	15

Table 2 shows that all three CKF outputs instantiate roughly the same shallow structure: seven entities, one concept, one contextual trigger, one mental model, one playbook, one QA pair, one retrieval chunk, one atomic unit and one traceability item. However, many semantically important CKF sections are empty, including principles, heuristics, decision rules, procedures, causal chains, if-then rules and exceptions. This is notable because the PDF explicitly discusses rules, exceptions, procedures, provenance and composition failures.

Table 3. Selected source knowledge units and preservation in the structured CKF artifacts.

Knowledge unit from PDF	PDF	Structured CKF result
Historical encoding of knowledge for human readers	Present	Absent
LLM agents as a new class of reader	Present	Partial
Composition failures as distinct from generic hallucination	Present	Absent
Implicit hierarchy, exception and procedural dependencies	Present	Absent
Move structural inference earlier through compilation	Present	Preserved
Typed package for agent consumption	Present	Preserved
Original PDF remains as a parallel artifact	Present	Absent
Compiler analogy: source code vs intermediate representation	Present	Absent
CKF composes with RAG, GraphRAG, vector stores, MCP and knowledge graphs	Present	Absent
Provenance and auditability as design commitments	Present	Partial/generic
Empirical confirmation through retrieval pressure, multi-hop questions, independent judge	Present	Absent
COMPGAP benchmark and falsifiability of CKF hypothesis	Present	Absent
MIT license, open implementation, public research	Present	Absent

The 20-unit rubric produced an estimated preservation score of approximately 17.5% for each structured CKF artifact. The strongest preserved units are the high-level idea that structural inference should move earlier and that a typed package can serve agent consumption. The weakest preserved units are the source’s arguments about composition failure, retrieval pressure, RAG/GraphRAG/MCP compatibility, benchmarking, falsifiability and public experimental evidence.

Table 4. Audit of the submitted CKF compiler code.

Subsystem	Observed design	Implication for preservation
Chunking	Semantic chunker; heading detection; stable spanId; approximately 12,000 characters per chunk; paragraphs used for sub-splitting.	Supports traceable per-span extraction but may under-segment short single-page material.
Map stage	Each chunk is sent to a provider with a tool-calling schema and a source wrapper.	Appropriate for structured extraction; output quality remains model- and prompt-dependent.
Schema	Traceable item schema requires source_basis, confidence, source_refs and source_excerpts for most item types.	Directly addresses provenance and attribution gaps found in the v0.1 artifacts.
Reducer	Map-reduce merger deduplicates entities, concepts, rules and chunks using normalized/fuzzy keys and builds source_traceability.	Improves package consolidation but may merge semantically distinct items if labels are similar.
Quality gates	Coverage, density, traceability, schema completeness and overall score are computed from populated sections and trace fields.	Useful for operational monitoring, but does not itself measure semantic fidelity to a gold source.
Provider layer	Multiple providers are normalized behind tool/function calling interfaces.	Portability is strong; reproducibility requires logging provider, model and version/date.

The code audit changes the interpretation of the empirical result. The structured artifacts analyzed here declare an older protocol pattern and are shallow, but the submitted compiler code targets CKF v1.0. The current schema requires trace fields for most extracted items, including confidence, source_refs and source_excerpts. The reducer builds source_traceability from span IDs and excerpts, and the quality module computes coverage, density, traceability and schema completeness. These mechanisms are directionally aligned with the very losses observed in the output artifacts.

6. Discussion

6.1 Format conversion is not knowledge compilation

The key result is that the structured artifacts gained computational form while losing much of the manifesto's argument. This distinction matters. A JSON object, YAML hierarchy or Markdown package can appear organized while containing only a thin semantic abstraction of the source. In this case, the packages contain useful containers - metadata, entities, concepts, chunks and instructions - but not enough of the source's explanatory structure.

This supports a central principle for CKF evaluation: schema completeness is necessary but insufficient. A populated schema section does not prove that important knowledge was preserved; an empty section does not necessarily prove that the source lacked that knowledge. The source in this case contained causal reasoning, design commitments and empirical hypotheses, but the CKF artifacts left the corresponding fields empty.

6.2 Why the observed CKF outputs are semantically shallow

Several factors may explain the shallow output. First, the source is short but conceptually dense; length-based extraction minimums may under-extract such documents. Second, the structured derivatives are in mixed languages, which may have introduced translation and alignment drift. Third, the analyzed artifacts appear to be generated under a different or older protocol than the submitted compiler code. Fourth, the evaluation corpus is a manifesto, whose knowledge is argumentative rather than procedural; a compiler must still capture principles, claims, causal relations and research commitments from such prose.

6.3 The compiler architecture is aligned with the research problem

The current compiler design is promising because it implements mechanisms that directly target the failure mode identified in the empirical artifacts. The semantic chunker assigns stable span IDs, the structured schema requests `source_basis` and `confidence`, and the `source_excerpts` requirement ties extracted claims to verbatim evidence. The reducer then converts per-chunk partials into a canonical package and reconstructs `source_traceability`. Quality gates make traceability and section population visible to users.

However, the current quality gates should be extended. Coverage, density, traceability and schema completeness are operational quality indicators, but they do not measure whether the most important claims from the source survived compilation. A CKF compiler should therefore include a semantic preservation score based on source-derived checklists, key-term retention and downstream retrieval behavior.

6.4 Implications for RAG, GraphRAG and agentic systems

RAG and GraphRAG systems depend on what is retrievable. If the indexed unit is a shallow CKF package, retrieval may be clean and structured but still miss the source's core argument. Conversely, a faithful CKF package could improve retrieval by making causal chains, exceptions, operational rules and provenance available as first-class objects. This is especially relevant where agents need to justify outputs, handle regulated domains or reason over procedural dependencies.

6.5 Recommendations for preservation-aware CKF compilation

Table 5. Recommendations for preservation-aware CKF compilation.

Recommendation	Rationale
Add a semantic preservation score	Compare generated CKF units against a gold checklist of source claims, not only populated schema sections.
Require <code>source_excerpts</code> for <code>core_intent</code> and <code>domain_map</code>	The current schema requires trace fields for most items but not verbatim excerpts for core intent.

Introduce minimums proportional to source length and genre	A short manifesto still contains many conceptual units; minimum thresholds based only on chunk length can under-extract dense prose.
Emit source-language packages by default	The prompt requires same language as the source; enforce this in tests to avoid silent translation drift.
Add key-term retention tests	Terms such as RAG, GraphRAG, provenance, composition failure and benchmark should be detected and retained.
Evaluate downstream retrieval	Measure whether CKF reduces composition errors under constrained top-k and context budgets.

7. Threats to Validity

The study has five important limitations. First, it is a single-document case study, so results should not be generalized without a larger corpus. Second, the preservation rubric was manually constructed, and other evaluators might weight knowledge units differently. Third, the structured artifacts and current compiler code appear to correspond to different protocol stages; this was treated as an empirical finding, but it limits causal attribution. Fourth, no live recompilation was performed with a fixed provider and model version in this paper. Fifth, legibility scores are ordinal expert judgments rather than results from a user study.

8. Conclusion

This study finds that converting a narrative PDF into structured CKF representations can dramatically improve computational legibility while preserving surprisingly little substantive knowledge if extraction is shallow. In the analyzed artifacts, JSON, Markdown and YAML preserve about 17.5% of a 20-unit knowledge rubric derived from the PDF. The missing information includes composition failures, provenance rationale, retrieval ecosystem positioning, benchmarking plans and falsifiability claims. The submitted compiler code, however, shows that CKF v1.0 has moved toward stronger auditability through `source_refs`, `source_excerpts`, confidence and quality scoring. The paper therefore concludes that the scientific question is not whether JSON, Markdown or YAML are better than PDF in isolation. The question is whether a compiler can preserve the right knowledge units, with evidence, in a form that agents can retrieve and reason over. Future CKF evaluation should combine schema validation, source-grounded semantic preservation and downstream retrieval benchmarks.

Data and Software Availability

The analyzed artifacts consist of the four CKF/PDF files and eight TypeScript compiler files supplied by the author for this study. A reproducibility package should include the original PDF, JSON, Markdown and YAML outputs; the compiler source code; the preservation rubric; and scripts used to compute file metrics and structural counts.

Conflict of Interest Statement

The author is affiliated with CKF Research, the organization developing the Compiled Knowledge Format. The empirical finding reported here is intentionally conservative and identifies limitations in the analyzed conversion artifacts.

References

- Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. International Conference on Learning Representations (ICLR). <https://openreview.net/forum?id=hSyW5go0v8>
- Bray, T. (2017). The JavaScript Object Notation (JSON) data interchange format (RFC 8259). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8259>
- CommonMark. (2024). CommonMark specification, version 0.31.2. <https://spec.commonmark.org/0.31.2/>
- Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024). From local to global: A Graph RAG approach to query-focused summarization. arXiv:2404.16130. <https://arxiv.org/abs/2404.16130>
- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2025). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. ACM Computing Surveys. <https://doi.org/10.1145/3703155>
- ISO. (2020). ISO 32000-2:2020, Document management - Portable document format - Part 2: PDF 2.0. International Organization for Standardization.
- JSON Schema. (2022). JSON Schema draft 2020-12. <https://json-schema.org/draft/2020-12>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459-9474. <https://arxiv.org/abs/2005.11401>
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. Transactions of the Association for Computational Linguistics, 12, 157-173. <https://aclanthology.org/2024.tacl-1.9/>
- Moreau, L., Groth, P., Cheney, J., Lebo, T., & Miles, S. (2015). The rationale of PROV. Journal of Web Semantics, 35, 235-257. <https://doi.org/10.1016/j.websem.2015.04.001>
- Rashkin, H., Nikolaev, V., Lamm, M., Aroyo, L., Collins, M., Das, D., Petrov, S., Tomar, G. S., Turc, I., & Reitter, D. (2023). Measuring attribution in natural language generation models. Computational Linguistics, 49(4), 777-840. https://doi.org/10.1162/coli_a_00490
- W3C Provenance Working Group. (2013). PROV-O: The PROV ontology. W3C Recommendation. <https://www.w3.org/TR/prov-o/>
- YAML Language Development Team. (2021). YAML Ain't Markup Language (YAML) version 1.2.2. <https://yaml.org/spec/1.2.2/>

Appendix A. Full preservation rubric used in the case study

Table A1. Unit-level preservation scoring.

#	Knowledge unit	PDF	JSON	MD	YAML
1	Knowledge historically encoded for human readers	1	0	0	0
2	LLM agents as a new class of reader	1	0.5	0.5	0.5
3	Mismatch between human documents and machine readers	1	0	0	0
4	Implicit relations require reconstruction at inference time	1	0	0	0
5	Composition failures distinguished from generic hallucination	1	0	0	0
6	CKF moves structural inference earlier	1	1	1	1
7	Rules, exceptions, procedures and provenance can be extracted once	1	0.5	0.5	0.5
8	Typed package/native machine consumption	1	1	1	1
9	Original PDF remains parallel artifact	1	0	0	0
10	Compiler analogy	1	0	0	0
11	CKF is not a replacement for RAG/GraphRAG/vector DB/MCP/KG	1	0	0	0
12	CKF composes with existing AI stack	1	0	0	0
13	Provenance through source basis, confidence and links	1	0.5	0.5	0.5
14	Auditability in regulated domains	1	0	0	0
15	Empirical uncertainty and need for confirmatory study	1	0	0	0
16	Retrieval pressure: context budget, top-k, multi-hop, independent judge	1	0	0	0
17	COMPGAP benchmark	1	0	0	0
18	Falsifiability of CKF hypothesis	1	0	0	0
19	MIT license/open implementation/open data	1	0	0	0
20	Closing slogan: narrative to structure/content to context	1	0	0	0